

Programowanie obiektowe i inżynieria oprogramowania

Lab AG-3:

- rozwiązanie zadania: klasa osobnika;
- klasa populacji

Akademia Górniczo-Hutnicza im. Stanisława Staszica w Krakowie
AGH University of Science and Technology

Prowadzący: dr inż. Jakub Bryła
2025

Klasa osobnika



Osobnik

Genotyp: $[x_1 x_2]$

Ocena: *double*

`rand_gens_val()`
`rate()`

Podmiana osobnika w przyszłości pozwoli na wykorzystanie algorytmu dla dowolnego rozwiązywanego zadania!

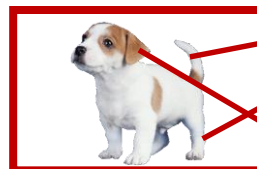
Co charakterystycznego będzie zawierał w sobie osobnik?

Geny – potrzebujemy:

- parametru składowego klasy - genotypu;
- funkcji do losowania wartości początkowej genów – tak powstaje pierwsza populacja.

Ocenę - potrzebujemy:

- funkcji oceny – f. oceny i definicja osobnika (jego genotyp) zależą bezpośrednio od rozwiązywanego zadania. **W osobniku będzie zdefiniowana funkcja oceny** - osobnik będzie więc jedyną zmienną w całym algorytmie, która zawiera w sobie wszystkie informacje, definiujące problem;
- parametru składowego do przechowywania wartości oceny;



dł. ogona
dł. łapy
liczba uszu

Gen

```
TParam.h  TCandidate.h  X
AG2024    TCandidate

1  #pragma once
2  #include "TParam.h"
3
4  #define GENS_COUNT 2
5
6  class TCandidate
7  {
8      TParam genotype[GENS_COUNT] =
9      {
10         TParam("x1", 0, 10, 1),
11         TParam("x2", 0, 10, 1)
12     };
13
14     double mark;
15
16 public:
17     TCandidate();
18
19     double get_mark() { return mark; };
20     void rate();
21
22     void info();
23
24 private:
25     void rand_gens_val();
26 }
```

W klasie osobnika wykorzystujemy klasę TParam.

Za pomocą instrukcji preprocesora definiujemy „stałą”, tj. liczbę genów (parametrów) osobnika.

W deklaracji klasy definiujemy zmienną – tablicę przechowującą wszystkie geny osobnika.

W deklaracji klasy definiujemy zmienną do przechowywania oceny osobnika.

Klasa zawiera cztery funkcje publiczne: konstruktor, f. zwracającą ocenę osobnika, f. do wyliczania oceny oraz f. wyświetlającą informacja o osobniku.

Funkcja do losowania wartości początkowej genów / parametrów osobnika

```

1  #include <iostream>
2  #include <math.h> // pow
3
4  #include "TCandidate.h"
5
6  using namespace std;
7
8  TCandidate::TCandidate()
9  {
10     mark = 0;
11     rand_gens_val();
12 }
13
14 void TCandidate::rate()
15 {
16     double x1 = genotype[0].get_val();
17     double x2 = genotype[1].get_val();
18
19     mark = pow(x1, 2) + x2;
20 }
21
22 void TCandidate::rand_gens_val()
23 {
24     for (int i = 0; i < GENS_COUNT; i++)
25     {
26         genotype[i].set_rand_val();
27     }
28 }
29
30 void TCandidate::info()
31 {
32     cout << "\n\n";
33     cout << "=====\n";
34     cout << "==" << "\n";
35     cout << "==" << "gens count: " << GENS_COUNT << "\n";
36
37     for (int i = 0; i < GENS_COUNT; i++)
38     {
39         cout << "==" << " " << genotype[i].get_name()
40             << " " << "\ value: " << genotype[i].get_val()
41             << "\n";
42     }
43
44     cout << "==" << "\n";
45     cout << "==" << " rate: " << mark << "\n";
46     cout << "=====\n\n";
47 }

```

Konstruktor odpowiedzialny jest za nadanie domyślnych wartości początkowym składowym parametrom klasy.

Pobranie wartości każdego genu: **wywoływana jest funkcja obiektu klasy Tgen!!!**

Definicja postaci funkcji oceny.

W pętli ustawiana jest losowa wartość dla każdego genu osobnika. **Wywoływana jest funkcja obiektu klasy Tgen!!!**

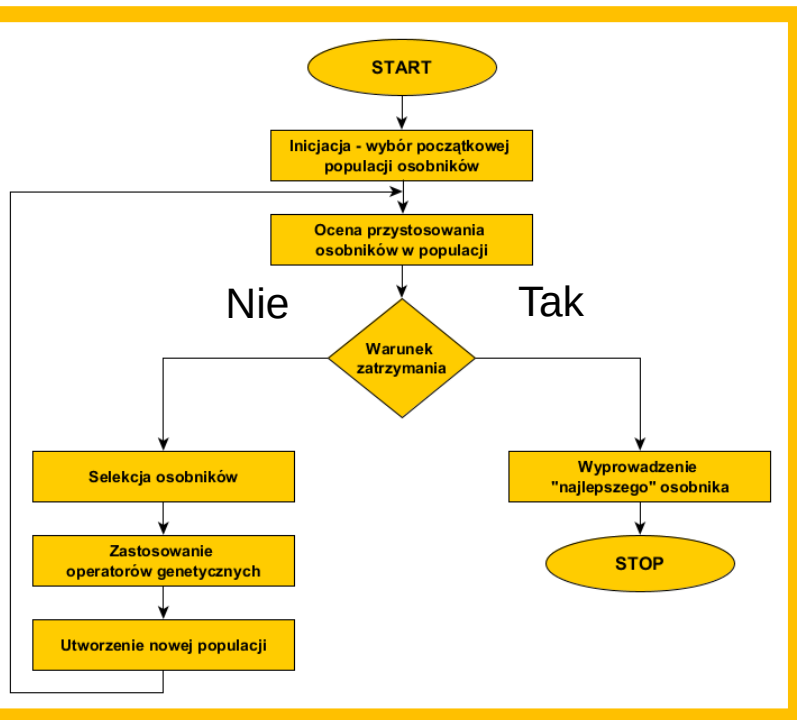
```
TCandidate.cpp  main.cpp  TParam.cpp
AG2024
1  #include <iostream>
2  #include <cstdlib> // srand
3  #include <time.h>
4
5  #include "TCandidate.h"
6
7  using namespace std;
8
9
10 int main()
11 {
12     srand(time(0));
13
14     TCandidate os1{};
15     os1.rate();
16     os1.info();
17
18     return 0;
19 }
```



```
=====
==
== gens count: 2
== "x1" value: 9
== "x2" value: 9
==
== rate: 90
=====
```

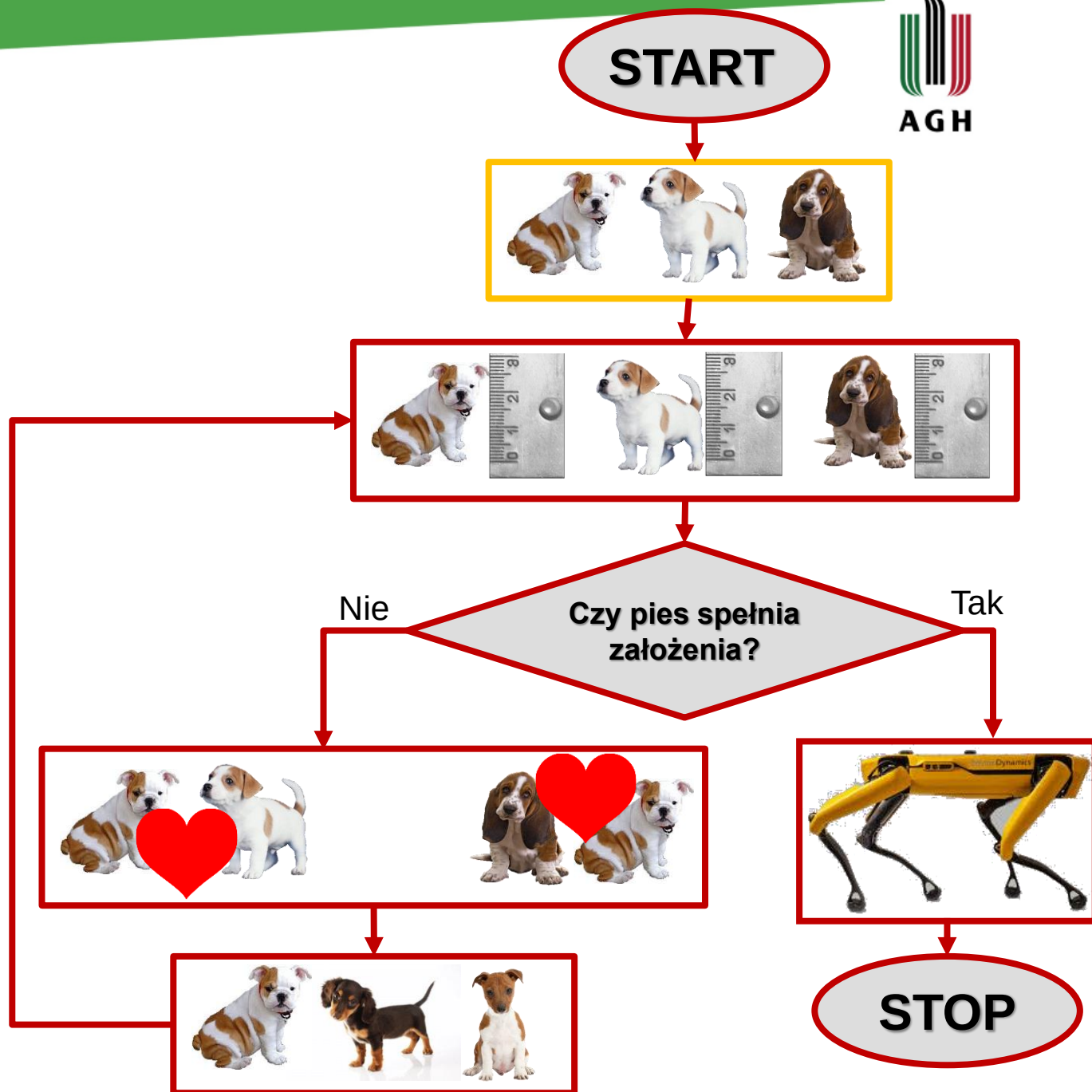
Klasa populacji

Algorytmy ewolucyjne



Funkcja oceny

wyznacza skalę spełnienia przez osobnika założonych wymagań





Populacja



Osobnik



dł. ogona
dł. łapy
liczba uszu

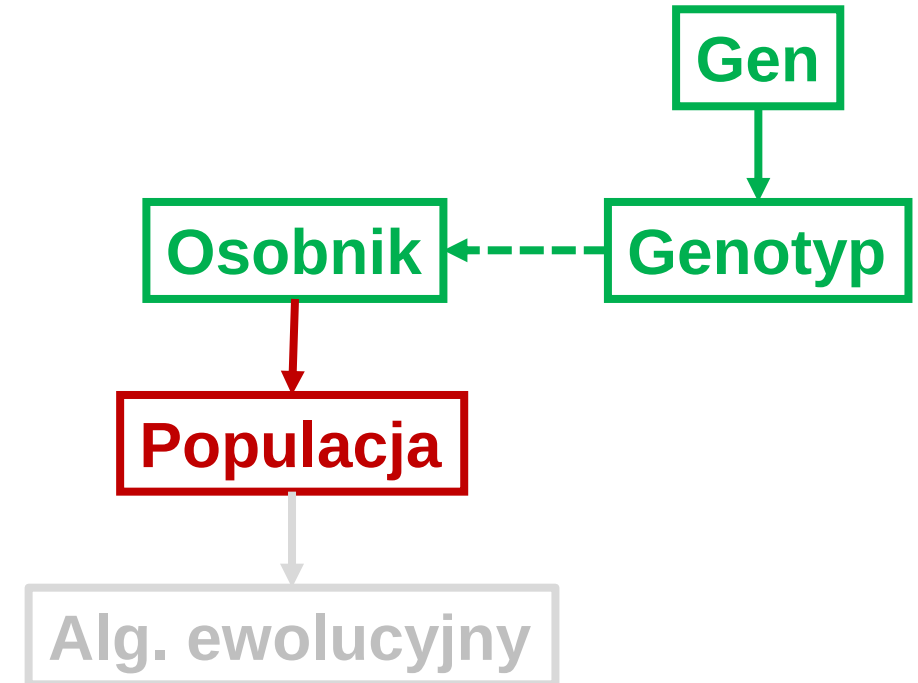
Gen

dł_ogona||dł_łapy||l_uszu

Genotyp

20cm||30cm||2sztuki

Fenotyp



Co charakterystycznego będzie zawierała w sobie populacja?

TPopulation



id: unsigned int
candidates_count: unsigned int
candidates: Tcandidate[]
best_val: double

void **calculate()**
Tcandidate **get_best_candidate()**

Tcandidate [] – symbolicznie zaznaczono wykorzystanie „pojemnika”, nie konieczne będzie to tablica...

Parametry składowe:

- **id** – ze schematu algorytmu genetycznego wynika, że w trakcie pracy algorytmu będzie tworzonych wiele populacji. Bardzo często wprowadza się warunek zatrzymania algorytmu po utworzeniu zadanej liczby populacji;
- **candidates_count** – liczba osobników w populacji;
- **candidates** – zbiornik na obiekty klasy Tcandidate;
- **best_val** – zmienna do przechowywania najlepszego wyniku.

Funkcje składowe:

- **calculate** – może się zdarzyć że obliczenie wartości funkcji oceny osobnika zajmuje sporo czasu. Zadanie to będzie wywołane w konkretnie zadanym momencie;
- **get_best_candidate** – funkcja umożliwiająca dalsze pobranie (wykorzystanie w innym miejscu kodu) najlepszego osobnika (finalnie został wybrany w jakimś celu...)

TPopulation



id: ~~_____~~ unsigned int
candidates_count: unsigned int
candidates: Tcandidate[]
best_val: double

void **calculate()**
Tcandidate **get_best_candidate()**

Tcandidate [] – symbolicznie
zaznaczono wykorzystanie
„pojemnika”, nie koniecznie
będzie to tablica...

**Na dzisiejszych zajęciach implementujemy kod klasy bez
parametru id!**

Efekt końcowy:

```
main.cpp x TCandidate.cpp TPopulation.h TParam.h
AG2024 (Globalny zasięg) main[
1 #include <iostream>
2 #include <cstdlib> // srand
3 #include <time.h>
4
5 #include "TPopulation.h"
6
7 using namespace std;
8
9
10 int main()
11 {
12     srand(time(0));
13
14     TPopulation pop(10);
15
16     cout << "Zaraz po utworzeniu obiektu klasy TPopulaiton";
17     pop.info();
18     cout << "\n\n";
19
20     cout << "Po wykonanych obliczeniach";
21     pop.calculate();
22     pop.info();
23     cout << "\n\n";
24
25     return 0;
26 }
```



```
Konsola debugowania programu Microsoft Visual Studio
Zaraz po utworzeniu obiektu klasy TPopulaiton

===== POPULATION #0 =====
== candidate#0: 0
== candidate#1: 0
== candidate#2: 0
== candidate#3: 0
== candidate#4: 0
== candidate#5: 0
== candidate#6: 0
== candidate#7: 0
== candidate#8: 0
== candidate#9: 0
=====

Po wykonanych obliczeniach

===== POPULATION #0 =====
== candidate#0: 90
== candidate#1: 9
== candidate#2: 101
== candidate#3: 64
== candidate#4: 106
== candidate#5: 90
== candidate#6: 24
== candidate#7: 54
== candidate#8: 90
== candidate#9: 16
=====
```

Linia 14: utworzenie populacji liczącej 10 osobników.

Linia 16 – 18: wyświetlenie informacji o populacji.

Linia 21: obliczenie wartości każdego osobnika populacji.

Linia 20 – 23: wyświetlenie informacji o populacji.

Programowanie obiektowe i inżynieria oprogramowania

Lab AG-3:

- rozwiązanie zadania: klasa osobnika;
- klasa populacji

Akademia Górniczo-Hutnicza im. Stanisława Staszica w Krakowie
AGH University of Science and Technology

Prowadzący: dr inż. Jakub Bryła
2025